

Gestion des Patients

Évaluation des Travaux

Rania Hammami

Patron de conception : Factory

- **Diagramme de classes** : Il manque les flèches entre `RendezVousCreator` et `RendezVous`, ce qui nuit à la compréhension des relations.
- Le **patron Factory est mal exploité** : au lieu de déléguer intelligemment la création à des sous-classes ou créateurs spécialisés, la classe `PfaFactory` contient des **conditions `if` imbriquées**, ce qui **ne met pas en valeur l'utilité du pattern**.
- **Conception** :
 - Une **classe abstraite** aurait dû être utilisée avec **redéfinition** de la méthode `createRendezVous()`, plutôt qu'une **interface**.
 - Il aurait été préférable de **séparer** la création des rendez-vous en ligne et présents dans deux **Concrete Factories distinctes** pour respecter les **principes SOLID** et améliorer la **clarté**.

GRASP – Polymorphisme

- L'application du polymorphisme est correcte.
- Cependant, **les attributs du rendez-vous** n'ont pas été intégrés dans le cadre du polymorphisme, ce qui aurait renforcé l'application du principe(`getDetails`).

SOLID – SRP/OCP

- Rania a mentionné les principes SOLID à travers l'application du Factory (elle a donné un exemple et une capture d'écran de l'architecture), mais nous

voudrions comprendre où se trouve exactement l'utilité et l'application du principe SRP même après avoir consulté le code

- Nous constatons que l'OCP est respecté, mais il manque des explications claires sur son application concrète, notamment à travers l'utilisation du patron Factory et du polymorphisme.

Contrainte OCL

- Elle a écrit `self.patient.rendezVous`, alors que cet attribut n'existe pas dans le diagramme de classes.

Skander Konte

Patron de conception : Bridge

- Le **patron Bridge** n'a pas été appliqué correctement.
- Les **flèches dans les diagrammes** sont incorrectes.
- Le **travail manque d'explications** claires du code réalisé.

GRASP – Information Expert

- Bon respect du principe GRASP :
 - Les **responsabilités** ont été attribuées aux classes **détenant réellement les données**.

SOLID – OCP

- Même si l'OCP semble appliqué à travers le Bridge, il n'a pas précisé où ni montré concrètement comment il l'a appliqué.

Contrainte OCL

- Le précondition `self.currentUser.role = UserRole::SECRETARY` manque de contexte d'application, notamment concernant l'endroit et la

manière dont cette condition OCL est intégrée dans le système, ainsi que les conséquences en cas d'échec de la vérification.

- La logique OCL doit être appliquée en cohérence avec le diagramme de classes. Actuellement, la contrainte `self.tableModel.rowCount() = Patient.allInstances()->size()` semble être définie dans un contexte d'interface sans prise en compte suffisante de la structure du diagramme de classes, ce qui crée une incohérence.

Amana Khachnaoui

Patron de conception : Observateur

- Application correcte du patron Observateur , mais c'est quoi l' utilité d'observer un contrôleur (Elle est en train de l'appliquer sur le contrôleur) hors que le contrôleur peut changer la méthode automatiquement.

GRASP - Controller

- Le controller prend les requêtes et transfère le travail à d'autres contrôleurs, mais quelle est l'utilité de cette approche ? Est-ce que cela pour une meilleure organisation ?

SOLID - SRP

- Le principe SRP a été correctement et clairement appliqué dans son code.

Contrainte OCL

- Pour une contrainte aussi simple (`self.generationDate <= Date::now()`), utiliser **Observer + Strategy + Factory + Decorator** est **surdimensionné**.

L'utilisation combinée de plusieurs patterns de conception (Factory, Observer, Strategy, Decorator) pour implémenter une simple contrainte de validation de date apparaît **excessive et injustifiée**.

Pour une vérification aussi élémentaire que "la date ne doit pas être dans le futur", une simple méthode de validation locale aurait suffi, sans alourdir inutilement l'architecture.

En effet :

- **Factory** : inutile ici car il n'y a pas une complexité ou une variabilité suffisante dans la création des validateurs.
- **Observer** : exagéré car il n'y a pas de changements fréquents ou complexes sur la date nécessitant une notification dynamique.
- **Strategy** : l'utilisation de plusieurs stratégies pour valider une date simple est surdimensionnée.
- **Decorator** : l'idée peut être tolérée pour ajouter plusieurs couches de validation, mais reste lourde dans ce cas précis.

⇒ La surutilisation de patterns pour une seule règle métier basique nuit à la **lisibilité, maintenabilité** et **efficacité** du code.

Une approche plus simple et directe aurait été **plus adaptée** ici.

Pour les questions 6+7

- Il y a un manque d'explication claire dans cette partie. En effet, l'utilisation du code SQL n'est pas justifiée, et aucun objectif ou raison n'est indiqué pour son inclusion. De plus, il n'y a pas de comparaison entre les résultats obtenus lors de la génération du MCD et le diagramme de classes initial.
- Pour la génération automatique du code Java, il manque encore des explications détaillées ou des comparaisons (les captures d'écran sont présentées sans analyse).

Enfin, il n'y a aucune mise en valeur des changements appliqués à travers les patrons de conception.

Remarques générales :

L'absence d'explications détaillées dans la partie OCL est notable : il n'y a ni exemples concrets, ni diagrammes de classes, ni tableaux précisant les contraintes, les attributs concernés, et les tables associées.

De manière générale, un grand manque d'explications est constaté dans plusieurs parties, ce qui engendre une ambiguïté quant à l'utilité et à l'application des différents éléments, qu'il s'agisse des patrons GOF, des principes SOLID, des responsabilités GRASP, ou encore des expressions OCL.

Cependant, on remarque un effort particulier dans la concentration sur les sections où les patrons sont appliqués, ce qui montre une certaine compréhension ciblée des concepts.

Remarque sur l'architecture

- L'équipe a utilisé **Java** , mais a aussi utilisé des **controllers** sans adopter de réelle **architecture MVC**.

⇒ Cela entraîne un **manque de cohérence** et rend l'**organisation du code difficile à suivre**.

- Il n'y a pas de diagramme de classes pour l'application de l'OCL, ce qui empêche de bien mettre en valeur son utilisation et d'assurer la cohérence avec la structure du système.

